

Report

Introduction

In this experiment, we compare the performance of two OpenCL convolution kernels - a naive version and an optimised version utilising local memory. The goal is to understand how memory access patterns and block sizes affect the execution time and efficiency of GPU kernels.

Memory Access Pattern and Synchronisation

Naive Kernel

- Reads all input values directly from global memory.
- High latency due to the slower access speed of global memory.
- Redundant reads when neighbouring work-items access overlapping regions.
- No synchronisation needed since each thread processes independently.

Local Memory Kernel

- Loads a tile of input data into faster shared (local) memory.
- Threads collaboratively load data into local memory.
- Requires synchronisation using barrier().
- Reduces redundant global memory reads but introduces overhead.

Performance Analysis for OpenCL Gaussian Blur Implementation

Test results show that the "optimised" kernel using local memory is slower than the naive implementation:

Kernel	Time (ms)	Work Group
Simple Kernel (8x8)	0.035	8x8
Simple Kernel (16x16)	0.035	16x16
Simple Kernel (16x8)	0.035	16x8
Optimized Kernel (8x8)	0.077	8x8
Optimized Kernel (16x16)	0.060	16x16
Optimized Kernel (16x8)	0.048	16x8

Speedup (optimized vs simple): 0.46x (meaning the optimised version is 2.2x slower)

Why Is the "Optimised" Version Slower?

Several factors could be causing this unexpected result:

1. Different Kernel Weights

In the original implementation:

- Naive kernel: Uses pre-divided weights (0.0625, 0.125, 0.25)

- Optimized kernel: Uses integer weights (1, 2, 4) with division by 16 at the end

This makes the kernels perform slightly different calculations, which could affect both accuracy and performance.

2. Local Memory Overhead

On modern GPUs like the Apple M1:

- The penalty for uncoalesced global memory access is much lower than on older GPUs
- The overhead of loading data into local memory and synchronising with `'barrier()'` can outweigh the benefits for small kernels
- The naive kernel might benefit from hardware texture caching that's already optimised for spatial locality

3. Redundant Operations

The optimised kernel performs several redundant operations:

- Calculating modulo operations for every pixel
- Performing boundary checks that may not be necessary for most pixels
- Using a more complex memory loading pattern

4. Small Image Size

With smaller images or when the kernel execution time is very short (microseconds), the overhead of kernel launch and synchronisation can dominate the actual computation time.

Conclusion

Although local memory can significantly improve performance for memory-bound operations, this benefit is

highly dependent on the problem size, access pattern, and hardware specifics. In this experiment, the naive

kernel was faster due to its simplicity and negligible synchronisation overhead.

Recommendations:

- Test on larger datasets or more complex convolution filters.
- Investigate whether increasing block size or using 2D convolution with padding helps.
- Profile memory access using tools like Intel VTune or Apple Instruments for deeper optimisation insights.